

The value calculated is 2,500 milliseconds. `True` means that the timer should only run once (generate only one event). We now need code in the event loop that checks for the event happening:

```
if event.type == TIMER_EVENT_ID:
    --- snip enable button, hide message ---
```

Since we specified `True` in the call to set the timer, this event is issued only once. If we want to repeat events every 2,500 milliseconds, we could set the last argument in the original call to `False` (or just let it default to `False`). To end repeated timer events, we would make a call to `set_timer()` and pass `0` (zero) as the second argument.

Building a Timer by Calculating Elapsed Time

The third approach for implementing a timer uses the current time as a starting point. We can then continuously query the current time and perform a simple subtraction to calculate the elapsed time. The code shown for this example runs in the main loop; later, we'll extract the timer-related code and build a reusable `Timer` class.

The `time` module of the Python Standard Library has this function:

```
time.time()
```

Calling this function returns the current time in seconds as a floating-point number. The value returned is the number of seconds that have passed since “epoch time,” which is defined as 00:00:00 UTC on January 1, 1970.

The code in [Listing 13-1](#) creates a timer by remembering the time when the user clicks Start. While the timer is running, we check in every frame to see if the desired amount of time has elapsed. You’ve already seen the user interface, so I’ll omit those details and some of the setup code for brevity.

File: InLineTimerExamples/ElapsedTime.py

```
# Timer in the main loop

--- snip ---
```